

Welcome

CS 472/572
Unsupervised Machine Learning

Clustering 1

Week-2 -Class -2

Spring 2026

Instructor: Dr. Khondker Fariha Hossain

k -means Clustering

- How to cluster categorical data?
- Variant of k -means is used for clustering categorical data: k -modes Method
 - Replace mean of cluster with mode of data
 - A new dissimilarity measures to deal with categorical objects
 - A frequency-based method to update modes of clusters.

Distance/Similarity Measures for Categorical Data

Instead of Euclidean, you use measures suited for categories:

1. **Hamming distance** → counts mismatches between categorical attributes.
2. **Jaccard similarity** → for binary/categorical presence–absence features.
3. **Matching coefficient** → % of attributes that match.
4. **Goodall's measure** or **Gower's similarity** → handle mixed data types (numeric + categorical).

Once you compute a similarity or distance matrix, you can feed it into clustering algorithms like **hierarchical clustering**.

K-Modes Algorithm

Adaptation of K-Means for categorical data

Designed to work where averaging (means) makes no sense.

Cluster centers = Modes

Each cluster is represented by its **mode** (the most frequent category in each feature).

Example: If most items in a cluster are "Red", "Circle", "Large", the cluster's center is (Red, Circle, Large).

Distance measure

Uses **Hamming distance** (counts the number of mismatched categories between a data point and the cluster mode).

Example: Comparing (Red, Circle, Large) VS (Blue, Circle, Small) → 2 mismatches.

Algorithm workflow

Randomly select k categorical points as initial modes.

Assign each data point to the closest mode (least mismatches).

Update cluster modes by taking the most frequent category in each attribute.

Repeat until assignments stabilize.

K-Modes Algorithm

1. Strengths

- Handles **large categorical datasets** efficiently.
- Results are **interpretable** (clusters described by actual categories).
- Robust compared to forcing categorical data into numeric form.

2. Applications

- Customer segmentation (demographics, preferences).
- Market basket analysis.
- Survey/medical records with categorical features

k -Medoids Clustering

- Picks actual objects to represent the clusters, using one representative object per cluster
- Each remaining object is clustered with the representative object which it is the most similar.
- Partitioning method is then performed based on the principle of **minimizing the sum of the dissimilarities** between each object and its assigned reference point
- Absolute Error criterion is used

Sum of absolute error $\nearrow$

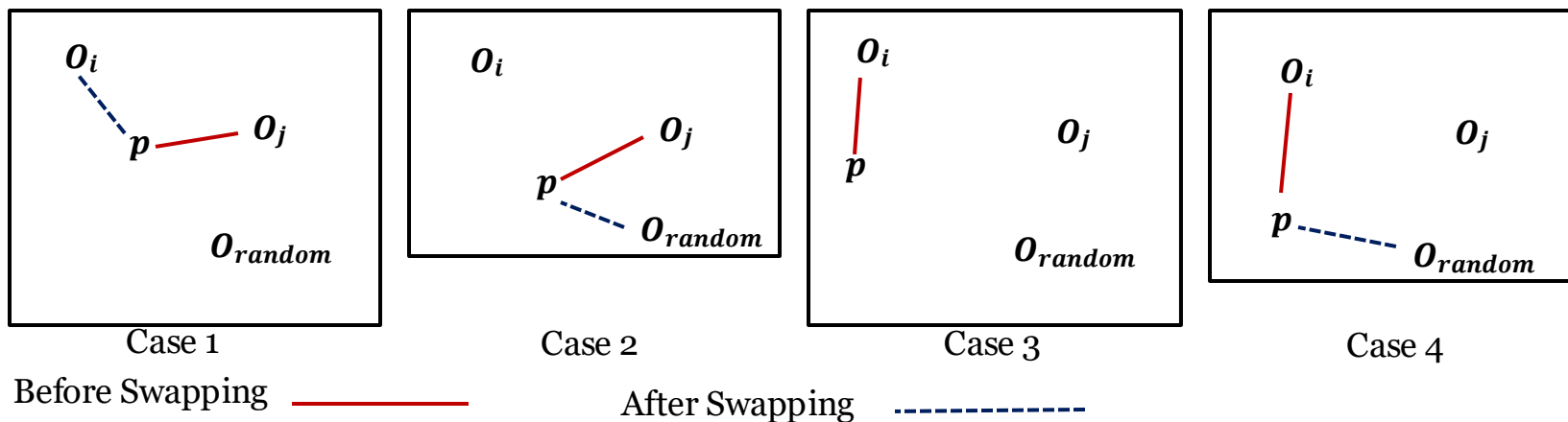
$$E = \sum_{j=1}^k \sum_{p \in c_j} \text{dist}(p, o_j)$$

Where

- p is the point in space representing a given object in cluster c_j
- o_j is the representative object of cluster c_j

k -Medoids Clustering

- The iterative process of replacing representative objects by nonrepresentative objects continues as long as the quality of the resulting clustering is improved.
- Quality is measured by a **cost function** that measures the average dissimilarity between an object and the representative object of its cluster.
- Four cases are examined for each of the nonrepresentative objects, p .
- Suppose, object p is currently assigned to a cluster represented by medoid o_j



k -Medoids Clustering

- Each time a reassignment occurs, a difference in absolute error, E , is contributed to the cost function.
- Therefore, the cost function calculates the difference in absolute-error value if a current representative object is replaced by a nonrepresentative object.
- The total cost of swapping is the sum of costs incurred by all nonrepresentative objects.
 - If the total cost is negative, then O_j is replaced or swapped with O_{random}
 - If the total cost is positive, the current representative object, O_j , is considered acceptable, and nothing is changed.
- PAM(Partitioning Around Medoids) was one of the first k -medoids algorithms

k -Medoids Clustering

Input: k number of clusters, n data objects from data set D

Output: a set of k clusters

Algorithm:

1. Arbitrarily select k objects as the representative objects or seeds
2. Repeat
 1. Assign each remaining objects to the cluster with the nearest representative object
 2. Randomly select the non- representative object O_{random}
 3. Compute the total cost S of swapping O_j with O_{random}
 4. If $S < 0$, then swap O_j with O_{random} to form the new set of k representative objects
3. Until no change

k -Medoids Clustering

Data Objects

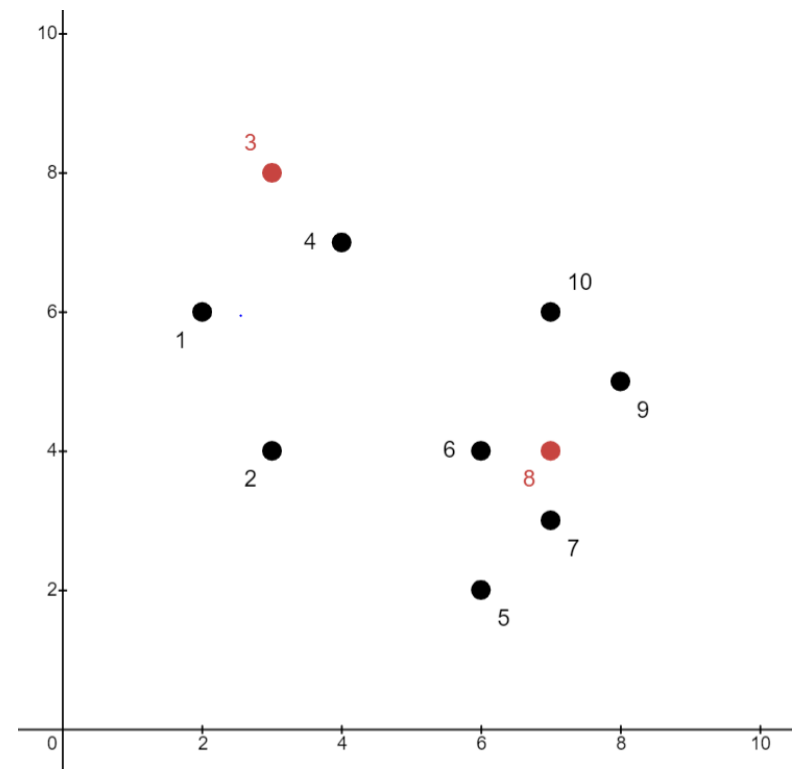
	X	Y
O1	2	6
O2	3	4
O3	3	8
O4	4	7
O5	6	2
O6	6	4
O7	7	3
O8	7	4
O9	8	5
O10	7	6

Aim: Create two Clusters

Step 1:
Choose randomly two medoids
(representative objects)

$$O_3 = (3, 8)$$

$$O_8 = (7, 4)$$



k -Medoids Clustering

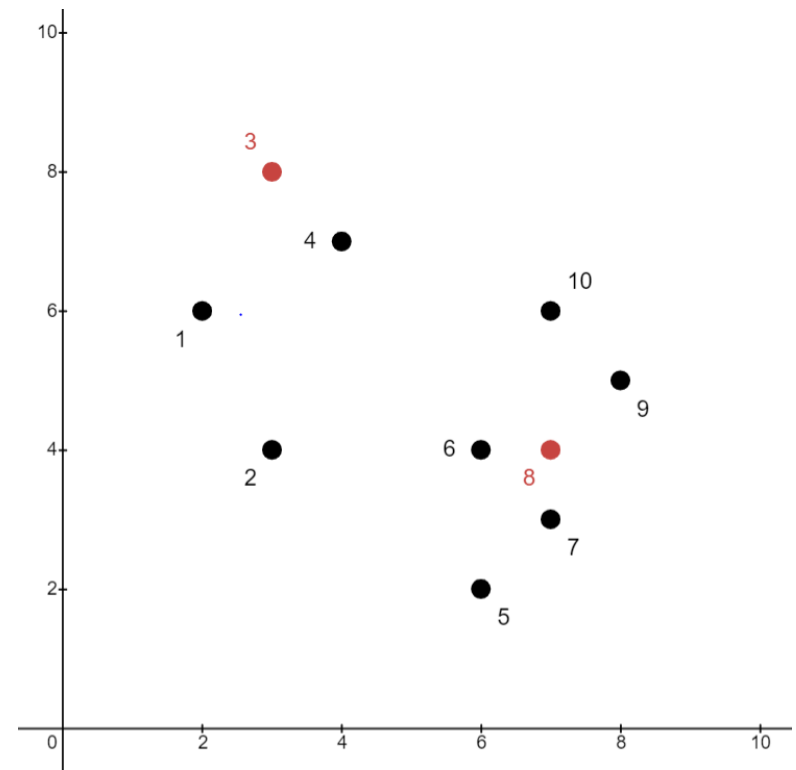
Data Objects

	X	Y	Cluster
O1	2	6	
O2	3	4	
O3	3	8	
O4	4	7	
O5	6	2	
O6	6	4	
O7	7	3	
O8	7	4	
O9	8	5	
O10	7	6	

Aim: Create two Clusters

Step 2:
Assign each object to the closest
representative object

Using Euclidean distance, we
form following clusters



k -Medoids Clustering

Data Objects

	X	Y	Cluster
O1	2	6	C1
O2	3	4	C1
O3	3	8	C1
O4	4	7	C1
O5	6	2	C2
O6	6	4	C2
O7	7	3	C2
O8	7	4	C2
O9	8	5	C2
O10	7	6	C2

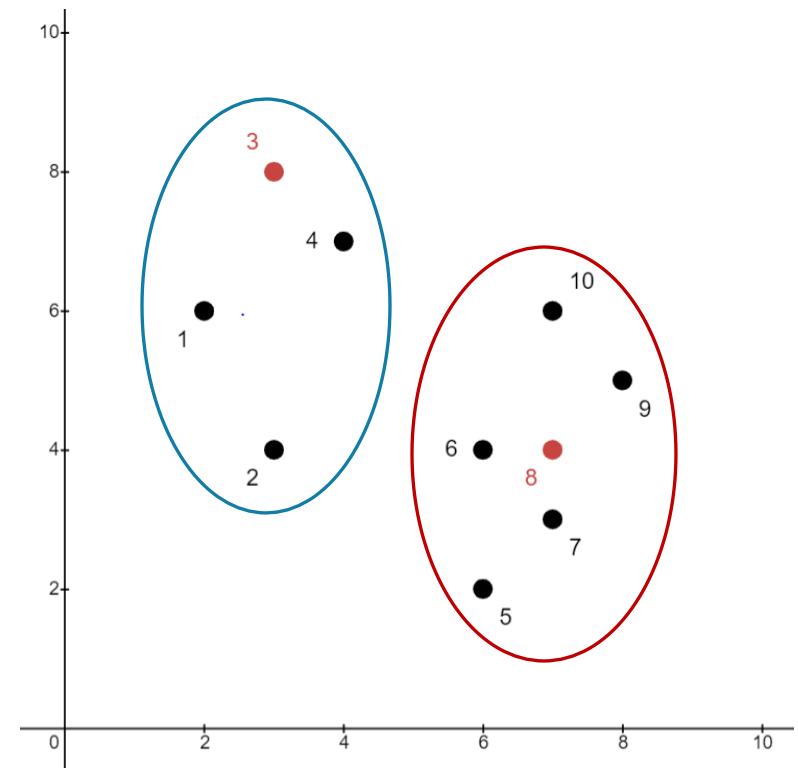
Aim: Create two Clusters

Step 2:
Assign each object to the closest
representative object

Using Euclidean distance, we
form following clusters

$C1 = \{O1, O2, O3, O4\}$

$C2 = \{O5, O6, O7, O8, O9, O10\}$



k -Medoids Clustering

Data Objects

	X	Y	Cluster
O1	2	6	C1
O2	3	4	C1
O3	3	8	C1
O4	4	7	C1
O5	6	2	C2
O6	6	4	C2
O7	7	3	C2
O8	7	4	C2
O9	8	5	C2
O10	7	6	C2

Aim: Create two Clusters

Step 3:

Compute the absolute error (for t

$$E = \sum_{j=1}^k \sum_{p \in c_j} |p - o_j|$$

$$E = |O_1 - O_3| + |O_2 - O_3| + |O_3 - O_3| + |O_4 - O_3| \\ + |O_5 - O_8| + |O_6 - O_8| + |O_7 - O_8| + |O_8 - O_8| + |O_9 - O_8| + |O_{10} - O_8|$$

$$|O_1 - O_3| = ||x_1 - x_3| + |y_1 - y_3|| \quad \dots \text{Manhattan Distance}$$

k -Medoids Clustering

Data Objects

	X	Y	Cluster
O1	2	6	C1
O2	3	4	C1
O3	3	8	C1
O4	4	7	C1
O5	6	2	C2
O6	6	4	C2
O7	7	3	C2
O8	7	4	C2
O9	8	5	C2
O10	7	6	C2

Aim: Create two Clusters

Step 3:

Compute the absolute error (for t

$$E = \sum_{j=1}^k \sum_{p \in c_j} |p - o_j|$$

$$E = |O_1 - O_3| + |O_2 - O_3| + |O_3 - O_3| + |O_4 - O_3| \\ + |O_5 - O_8| + |O_6 - O_8| + |O_7 - O_8| + |O_8 - O_8| + |O_9 - O_8| + |O_{10} - O_8|$$

$$E = (3 + 4 + 0 + 2) + (3 + 1 + 1 + 0 + 2 + 2)$$

$$E = 18$$

k -Medoids Clustering

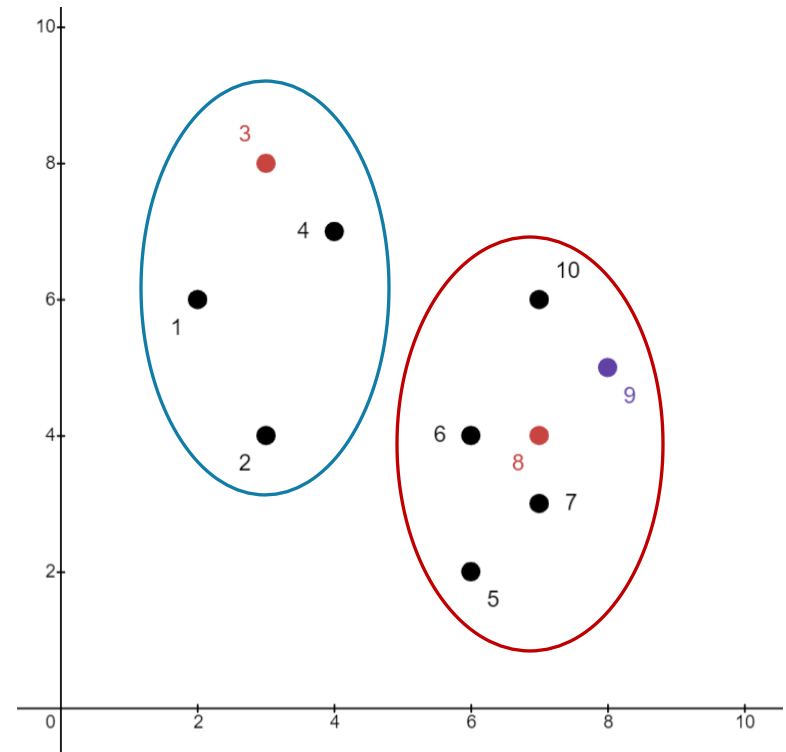
Data Objects

	X	Y	Cluster
O1	2	6	C1
O2	3	4	C1
O3	3	8	C1
O4	4	7	C1
O5	6	2	C2
O6	6	4	C2
O7	7	3	C2
O8	7	4	C2
O9	8	5	C2
O10	7	6	C2

Aim: Create two Clusters

Step 4:
Choose a random object O_9
Swap O_8 and O_9

Compute the absolute error (for
the set of representative objects
 O_3 and O_9)



k-Medoids Clustering

Data Objects

	X	Y	Cluster
O1	2	6	C1
O2	3	4	C1
O3	3	8	C1
O4	4	7	C1
O5	6	2	C2
O6	6	4	C2
O7	7	3	C2
O8	7	4	C2
O9	8	5	C2
O10	7	6	C2

Aim: Create two Clusters

$$E = |O_1 - O_3| + |O_2 - O_3| + |O_3 - O_3| + |O_4 - O_3| + |O_5 - O_9| + |O_6 - O_9| + |O_7 - O_9| + |O_8 - O_9| + |O_9 - O_9| + |O_{10} - O_9|$$

$$E = (3 + 4 + 0 + 2) + (5 + 3 + 3 + 2 + 0 + 2)$$

$$E = 24$$

Step 5:

Compute the cost function

$$S = \text{Absolute Error}(\text{for } O_3, O_8) - \text{Absolute Error}(\text{for } O_3, O_9)$$

$$S = 18 - 24 = -6$$

As $S < 0$, we swap O_8 with O_9

k -Medoids Clustering

Data Objects

	X	Y	Cluster
O1	2	6	C1
O2	3	4	C1
O3	3	8	C1
O4	4	7	C1
O5	6	2	C2
O6	6	4	C2
O7	7	3	C2
O8	7	4	C2
O9	8	5	C2
O10	7	6	C2

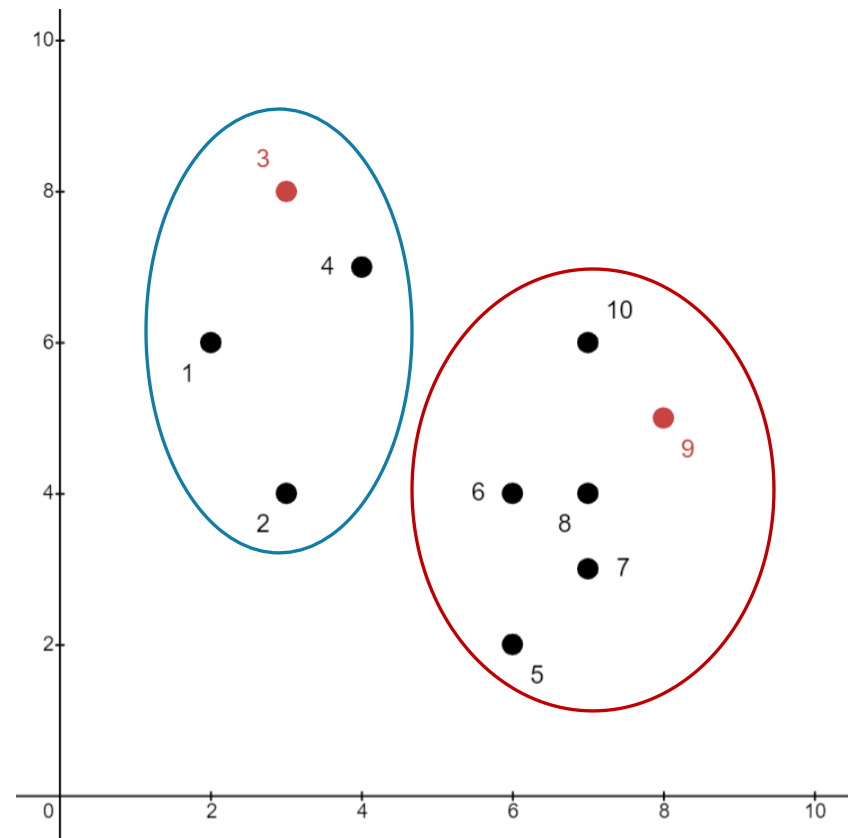
Aim: Create two Clusters

Step 6:

New medoids are O_3 with O_9

Repeat Step 2

Assign each object to the closest representative object.



k -Medoids Clustering

- Which method is more robust k -Means or k -Medoids?
 - The k -medoids method is more robust than k -means in the presence of noise and outliers, because a medoid is less influenced by outliers or other extreme values than a mean.
- The processing of k -Medoids is more costly than the k -means method.

Properties of K-means algorithm

- Guaranteed to converge in a finite number of iterations
- Running time per iteration:
 1. Assign data points to closest cluster center
 $O(KN)$ time
 2. Change the cluster center to the average of its assigned points
 $O(N)$

Example: K-Means for Segmentation

K=2



Goal of Segmentation is to partition an image into regions each of which has reasonably homogenous visual appearance.

Original



Example: K-Means for Segmentation

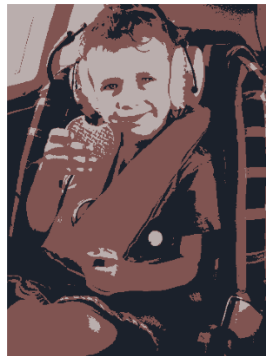
K=2



K=3



Original



Example: K-Means for Segmentation

K=2



K=3



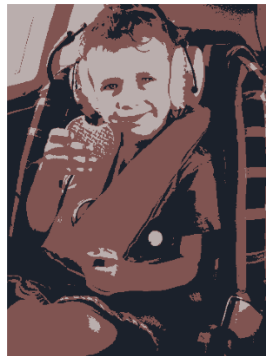
K=10



Original



4%



8%



17%



k-means Vs K-medoids Vs K-modes

Aspect	K-means	K-medoids	K-modes
Data type	Numeric/continuous features	Any type for which a distance is defined (numeric, categorical, string, edit distance, etc.)	Categorical features (discrete labels)
Cluster “center”	Centroid = mean of points (not necessarily a real data point)	Medoid = actual data point minimizing total distance to others	Mode = most frequent category per feature (component-wise)
Dissimilarity	Typically Euclidean (squared distance)	Arbitrary distance (Euclidean, Manhattan, DTW, edit distance...)	Matching/Hamming dissimilarity (0 if equal, 1 if not)
Objective	Minimize sum of squared distances (intra-cluster variance)	Minimize sum of distances to medoids	Minimize sum of mismatches to modes
Outliers	Sensitive (means get pulled)	Robust(er) (medoid is a real point, less pulled)	Robust to numeric outliers (not used); can be sensitive to rare categories
Shape assumption	Works best for spherical , similarly sized clusters after scaling	More flexible (depends on chosen distance)	Categorical analog of k-means; no geometric shape assumption
Per-iteration cost (rough)	$O(nkd)$ + updates	Higher than k-means; PAM $\sim O(k(n-k)^2)$ (CLARA/CLARANS are faster approximations)	$O(nkp)$ (p = #categorical features)
Initialization	Often k-means++	Medoid heuristics or k-means++-style on distances	Random modes or k-modes++
Common variants	Mini-batch k-means, kernel k-means	PAM, CLARA, CLARANS	k-modes with frequency weights; k-prototypes (for mixed data)

When to use which!!!

- ✓ **Use K-means** when features are numeric, scaled, and you expect roughly compact (spherical) clusters. Fast and simple; watch out for outliers and scaling.
- ✓ **Use K-medoids** when you want robustness or need a **custom distance** (e.g., Manhattan, DTW for time series, edit distance for strings) and/or you want a **representative example** as the center.
- ✓ **Use K-modes** when features are **purely categorical** (e.g., color = {red, blue}, brand = {A,B,...}).
For **mixed numeric + categorical**, use **k-prototypes** (modes for categorical + means for numeric).

Practical tips

- Always **standardize/normalize numeric features** before K-means/K-medoids (if using numeric distances).
- Try several **k** and seeds; evaluate with internal indices (e.g., silhouette) and **stability checks**.
- For large **n** with K-medoids, prefer **CLARA/CLARANS** (sampling/heuristic) to keep runtime manageable.

Example: Vector quantization



FIGURE 14.9. *Sir Ronald A. Fisher (1890 – 1962) was one of the founders of modern day statistics, to whom we owe maximum-likelihood, sufficiency, and many other fundamental concepts. The image on the left is a 1024×1024 grayscale image at 8 bits per pixel. The center image is the result of 2×2 block VQ, using 200 code vectors, with a compression rate of 1.9 bits/pixel. The right image uses only four code vectors, with a compression rate of 0.50 bits/pixel*

[Figure from Hastie *et al.* book]

What is Vector Quantization?

Vector Quantization = Clustering for compression

1. Technique that maps a large set of vectors into a smaller set of representative vectors
2. These representative vectors are called codebook vectors (or prototypes)
3. Purpose: reduce data complexity and storage while preserving structure
4. Widely used in compression, clustering, and pattern recognition

How It Works

1. Training phase: Cluster data (e.g., with K-Means) to create codebook vectors
2. Encoding: Replace each data point with nearest code vector's index
3. Decoding: Reconstruct data by replacing index with its code vector
4. Compact representation with some information loss

- *VQ is essentially K-Means clustering repurposed for compression*
- *Cluster centers = codebook vectors*
- *Each data vector is quantized to nearest codebook vector*

Application

- Imagine millions of 2D points
 - Instead of storing all, cluster into 100 regions
 - Each region is represented by its centroid (a code vector)
 - Store 100 centroids + assignments instead of all points
-
- - Image compression (reduce pixel values into representative colors)
 - - Speech coding (audio frames represented with codebook sounds)
 - - Pattern recognition (simplify feature spaces)
 - - Neural networks (product quantization in ANN search)

On Class EXERCISE

Link: <https://colab.research.google.com/drive/1o-NvOvMF10W6u3FNGegx-dUKeXmYr3sg?usp=sharing>

Thank You

Reference:

Credits: Dr.R.R.Rajalaxmi Professor/CSE Kongu Engineering College Perundurai
Ms. Rashmi Bhat, Assistant prof, Slideshare

David Sontag New York University

Dr. T.Veeramakali, Assoc. Prof. / DSBS, SRM Institute of Science & Technology
Kattankulathur, Chennai-603203.

.